Nested Kriging for Time Series Forecasting

Foundations, Adaptive Kernels, Missing Data,
Multivariate Extension, and Cross-Dataset Transfer

July 7, 2026

Contents

1 Introduction **3**

2 Window-Based Supervised Forecasting **3**
2.1 Window Pooling from a Univariate Series . . . 3
2.2 Normalisation . . . 3
2.3 Forecasting at Test Time . . . 3

3 Gaussian Process Regression **4**
3.1 Prior and Likelihood . . . 4
3.2 Multi-Output Targets . . . 4
3.3 Scalability Bottleneck . . . 4

4 Nested Kriging **4**
4.1 Overview . . . 4
4.2 Block Partitioning . . . 4
4.3 Local Block Prediction . . . 5
4.4 Second-Layer Aggregation . . . 5

5 Adaptive Lengthscales: The Gibbs Kernel **6**
5.1 Motivation . . . 6
5.2 Process Convolution Derivation . . . 6

6 Learning the Lengthscale Field **7**
6.1 Two-Layer Architecture . . . 7
6.2 Stage 1: Local Lengthscale Estimation via MLE . . . 7
6.3 Stage 2: Meta-GP over Block Centroids . . . 8
6.4 Stage 3: Jensen-Corrected Lengthscale Prediction . . . 8
6.5 Using the Lengthscale Field in the Gibbs Kernel . . . 9

7 Missing and Unobserved Data **9**
7.1 Types of Missingness in Time Series . . . 9
7.2 Sporadic Missingness: Linear Imputation and Filtering . . . 9
7.3 Intermittent Demand: Two-Model Approach . . . 9

8 Multivariate Time Series **10**
8.1 Problem Setup . . . 10
8.2 Channel-by-Channel Window Pooling . . . 10

8.3 Per-Channel NestedKrigingForecasters 11
8.4 MultivariateNestedKrigingForecaster 11

9 Cross-Dataset Nested Kriging **11**
9.1 Motivation and Challenge 11
9.2 Feature Map Approach (Warped Kernel) 12
9.3 Class-Level Intermediate Aggregation 12
9.4 Full Cross-Dataset Architecture 13

10 Summary and Notation Index **13**
10.1 Summary of the Full Method 13
10.2 Notation Index 14
10.3 References 14

1 Introduction

This document gives a self-contained derivation of the nested kriging forecasting system developed for the GIFT-Eval benchmark. The method treats univariate and multivariate time series forecasting as a supervised regression problem over sliding context windows and addresses four progressively harder settings:

1. **Within-dataset forecasting** with complete data.
2. **Within-dataset forecasting** with missing or intermittent observations.
3. **Multivariate** series sharing a common channel structure.
4. **Cross-dataset transfer**, aggregating evidence across incompatible series.

In each setting the workhorse is *nested kriging* (Rullière et al., 2018) — a scalable approximation to full Gaussian process regression that avoids forming the global $N \times N$ kernel matrix by partitioning training windows into blocks and combining local block predictors via a second-layer GP. On top of this we place a *Gibbs kernel* whose position-dependent lengthscale $\ell(x)$ is itself learned by a nested kriging meta-model, yielding a two-layer architecture that adapts to heterogeneous temporal dynamics without any parametric assumptions about how patterns vary across the context window space.

2 Window-Based Supervised Forecasting

2.1 Window Pooling from a Univariate Series

Let $\{y_t\}_{t=1}^{T}$ be a scalar time series. Fix a context length $L \in \mathbb{N}$ and a horizon $H \in \mathbb{N}$. For each admissible time index $t \in \{L+1, \ldots, T-H\}$ define the context–target pair

$$\mathbf{x}_t = (y_{t-L},\, y_{t-L+1},\, \ldots,\, y_{t-1}) \in \mathbb{R}^L, \qquad \mathbf{f}_t = (y_t,\, y_{t+1},\, \ldots,\, y_{t+H-1}) \in \mathbb{R}^H. \tag{1}$$

The full training set from a dataset containing S series is obtained by pooling all valid pairs across all series:

$$\mathcal{D} = \big\{(\mathbf{x}_i,\, \mathbf{f}_i)\big\}_{i=1}^{N}, \qquad \mathbf{X} \in \mathbb{R}^{N \times L}, \quad \mathbf{Y} \in \mathbb{R}^{N \times H}. \tag{2}$$

A stride parameter controls how densely windows are sampled, and a per-series cap prevents any single long series from dominating $\mathcal{D}$.

2.2 Normalisation

Each series is normalised before pooling to place all windows in a comparable scale. The *robust-series* normalisation subtracts the series median $\tilde{\mu}$ and divides by the inter-quartile range σ_{IQR}:

$$y'_t = \frac{y_t - \tilde{\mu}}{\sigma_{\mathrm{IQR}} + \varepsilon}, \qquad \varepsilon > 0 \text{ (numerical floor)}. \tag{3}$$

Series with zero IQR (e.g. constant segments) fall back to z-score normalisation. After prediction, outputs are de-normalised using the stored $(\tilde{\mu}, \sigma_{\mathrm{IQR}})$ of the context window.

2.3 Forecasting at Test Time

At test time, the most recent L observations of a held-out series form a query context $\mathbf{x}^* \in \mathbb{R}^L$. The model returns a predicted trajectory $\hat{\mathbf{f}}^* \in \mathbb{R}^H$ together with a scalar predictive variance $\hat{\sigma}^{*2}$.

3 Gaussian Process Regression

3.1 Prior and Likelihood

A Gaussian process $f \sim \mathcal{GP}(\mu_0, k)$ is fully specified by its mean function $\mu_0 : \mathbb{R}^L \to \mathbb{R}$ (taken to be zero here) and a symmetric positive semi-definite *covariance kernel* $k : \mathbb{R}^L \times \mathbb{R}^L \to \mathbb{R}$. Given n training points $\mathbf{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_n)^\top$ with noisy scalar targets $\mathbf{y} = (y_1, \ldots, y_n)^\top$,

$$y_i = f(\mathbf{x}_i) + \varepsilon_i, \quad \varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_n^2), \tag{4}$$

the posterior at a new point $\mathbf{x}^*$ is Gaussian:

$$\mu(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{X})\big[\mathbf{K} + \sigma_n^2 \mathbf{I}\big]^{-1} \mathbf{y}, \tag{5}$$

$$\sigma^2(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - k(\mathbf{x}^*, \mathbf{X})\big[\mathbf{K} + \sigma_n^2 \mathbf{I}\big]^{-1} k(\mathbf{X}, \mathbf{x}^*), \tag{6}$$

where $[\mathbf{K}]_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is the $n \times n$ training kernel matrix.

3.2 Multi-Output Targets

When targets are H-dimensional, treat each output dimension $h \in \{1, \ldots, H\}$ independently. Because the GP prior is shared and only the target column changes, the Cholesky decomposition of $\mathbf{K} + \sigma_n^2 \mathbf{I}$ is computed once and reused for all H right-hand sides. The mean prediction is then

$$\hat{\mathbf{Y}} = k(\mathbf{x}^*, \mathbf{X})\big[\mathbf{K} + \sigma_n^2 \mathbf{I}\big]^{-1} \mathbf{Y} \in \mathbb{R}^H, \tag{7}$$

and $\sigma^2(\mathbf{x}^*)$ from (6) serves as a single scalar uncertainty estimate common to all horizons.

3.3 Scalability Bottleneck

Full GP regression costs $\mathcal{O}(N^3)$ in time and $\mathcal{O}(N^2)$ in memory due to the Cholesky decomposition of the $N \times N$ matrix $\mathbf{K}$. For GIFT-Eval datasets, N can reach 10^4–10^6 pooled windows, making exact GP regression infeasible. Nested kriging resolves this by partitioning $\mathcal{D}$ into $M \ll N$ blocks and working exclusively with $\mathcal{O}(n_m^2)$ local matrices.

4 Nested Kriging

4.1 Overview

Nested kriging (Rullière et al., 2018) approximates the global GP posterior by:

1. Partitioning the N training windows into M spatial blocks.
2. Fitting a local GP independently within each block.
3. Aggregating the M local predictors via a second-layer GP whose "observations" are the local block predictions at each block centroid.

No $N \times N$ matrix is ever formed. The dominant cost is $\mathcal{O}(M n_{\max}^3)$ where $n_{\max} = \max_m n_m$ is the largest block size.

4.2 Block Partitioning

Definition 4.1 (Block partition)**.** A block partition of $\{1, \ldots, N\}$ is a family $\{B_1, \ldots, B_M\}$ of disjoint non-empty index sets with $\bigcup_{m=1}^{M} B_m = \{1, \ldots, N\}$.

In practice blocks are formed by randomly shuffling the N windows and cutting them into chunks of target size n_{block}. This randomised partition avoids any spatial bias that would arise from time-ordered chunking, which would confound temporal position with block identity.

The *centroid* of block m is

$$\bar{\mathbf{x}}_m = \frac{1}{n_m} \sum_{i \in B_m} \mathbf{x}_i \in \mathbb{R}^L, \tag{8}$$

and serves as the representative input for block m in the second-layer aggregation.

4.3 Local Block Prediction

For query $\mathbf{x}^*$, block m produces a local GP prediction using only its own n_m training pairs:

$$\hat{\mu}_m(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{X}_m)\big[\mathbf{K}_m + \sigma_n^2 \mathbf{I}_{n_m}\big]^{-1} \mathbf{Y}_m \in \mathbb{R}^H, \tag{9}$$

$$\hat{s}_m^2(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - k(\mathbf{x}^*, \mathbf{X}_m)\big[\mathbf{K}_m + \sigma_n^2 \mathbf{I}_{n_m}\big]^{-1} k(\mathbf{X}_m, \mathbf{x}^*) \in \mathbb{R}, \tag{10}$$

where $\mathbf{X}_m \in \mathbb{R}^{n_m \times L}$ collects the contexts in block m and $\mathbf{Y}_m \in \mathbb{R}^{n_m \times H}$ collects their targets.

4.4 Second-Layer Aggregation

The M local predictors are combined by modelling them as noisy observations of an underlying meta-GP $g \sim \mathcal{GP}(0, k)$ (same kernel as the local GPs):

$$\hat{\mu}_m(\bar{\mathbf{x}}_m) = g(\bar{\mathbf{x}}_m) + \eta_m, \quad \eta_m \sim \mathcal{N}(0,\, \hat{s}_m^2(\bar{\mathbf{x}}_m)). \tag{11}$$

The noise variance $\hat{s}_m^2(\bar{\mathbf{x}}_m)$ is the local prediction uncertainty at the centroid of block m, and reflects how well block m can represent a point at its own centre.

Define

$$\boldsymbol{c}(\mathbf{x}^*) = \big(k(\mathbf{x}^*, \bar{\mathbf{x}}_1), \ldots, k(\mathbf{x}^*, \bar{\mathbf{x}}_M)\big)^\top \in \mathbb{R}^M, \tag{12}$$

$$\mathbf{C} = \big[k(\bar{\mathbf{x}}_m, \bar{\mathbf{x}}_{m'})\big]_{m,m'} \in \mathbb{R}^{M \times M}, \tag{13}$$

$$\mathbf{D} = \operatorname{diag}\big(\hat{s}_1^2(\bar{\mathbf{x}}_1), \ldots, \hat{s}_M^2(\bar{\mathbf{x}}_M)\big) \in \mathbb{R}^{M \times M}. \tag{14}$$

The nested kriging (NK) predictor is

$$\hat{\mathbf{f}}_{\mathrm{NK}}(\mathbf{x}^*) = \boldsymbol{c}(\mathbf{x}^*)^\top \big[\mathbf{C} + \mathbf{D}\big]^{-1} \hat{\boldsymbol{\mu}}, \tag{15}$$

where $\hat{\boldsymbol{\mu}} = (\hat{\mu}_1(\bar{\mathbf{x}}_1), \ldots, \hat{\mu}_M(\bar{\mathbf{x}}_M))^\top \in \mathbb{R}^{M \times H}$ stacks the local predictions at the centroids. The product in (15) broadcasts over the H columns.

The corresponding NK predictive variance is

$$\hat{\sigma}_{\mathrm{NK}}^2(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - \boldsymbol{c}(\mathbf{x}^*)^\top \big[\mathbf{C} + \mathbf{D}\big]^{-1} \boldsymbol{c}(\mathbf{x}^*). \tag{16}$$

Remark 4.1 (Computational cost)**.** The second-layer solve requires factorising $\mathbf{C} + \mathbf{D} \in \mathbb{R}^{M \times M}$, costing $\mathcal{O}(M^3)$. With $M \sim 50$–200 blocks, this is negligible. The dominant cost is the M local Cholesky factorisations, each $\mathcal{O}(n_{\mathrm{block}}^3)$ with $n_{\mathrm{block}} \sim 20$–$50$.

Remark 4.2 (Interpretation)**.** Equation (15) is exactly standard GP regression of the M centroid predictions against the meta-kernel k, with noise $\mathbf{D}$ encoding local uncertainty. Blocks with high local uncertainty receive lower effective weight; blocks whose centroid is close to $\mathbf{x}^*$ in kernel distance receive higher weight.

5 Adaptive Lengthscales: The Gibbs Kernel

5.1 Motivation

The standard squared-exponential (RBF) kernel

$$k_{\mathrm{SE}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{\ell^2}\right) \tag{17}$$

uses a single global lengthscale ℓ that is the same everywhere in $\mathbb{R}^L$. For time series windows this is a poor fit: seasonal patterns at one part of the context window may be highly informative while other lags carry little signal. A single ℓ cannot simultaneously capture sharp local transitions and long smooth trends.

The *Gibbs kernel* replaces the scalar ℓ with a field $\ell : \mathbb{R}^L \to \mathbb{R}_{>0}$ that can vary smoothly over the window space. This lets the kernel concentrate correlation on informative regions of $\mathbb{R}^L$ and relax it elsewhere.

5.2 Process Convolution Derivation

Definition 5.1 (Gibbs kernel, Gibbs 1997)**.** Let $\ell : \mathbb{R}^L \to \mathbb{R}_{>0}$ be a smooth positive function. The Gibbs kernel is

$$k_{\mathrm{G}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(\frac{2\,\ell(\mathbf{x})\,\ell(\mathbf{x}')}{\ell(\mathbf{x})^2 + \ell(\mathbf{x}')^2}\right)^{1/2} \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{\ell(\mathbf{x})^2 + \ell(\mathbf{x}')^2}\right). \tag{18}$$

The Gibbs kernel arises naturally from a *process convolution* construction. Define

$$f(\mathbf{x}) = \int_{\mathbb{R}} h_{\mathbf{x}}(t)\, \mathrm{d}W(t), \tag{19}$$

where W is a standard Wiener process and $h_{\mathbf{x}} : \mathbb{R} \to \mathbb{R}$ is a position-dependent filter centred at $\mathbf{x}$. Choose a Gaussian filter:

$$h_{\mathbf{x}}(t) = \exp\left(-\frac{(t - \mathbf{x})^2}{2\,\ell(\mathbf{x})^2}\right), \tag{20}$$

so that the filter at position $\mathbf{x}$ has bandwidth $\ell(\mathbf{x})$ (here $\mathbf{x} \in \mathbb{R}$ for clarity; the multidimensional case proceeds componentwise). The covariance of f is

$$\begin{aligned} k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[f(\mathbf{x})\, f(\mathbf{x}')] &= \int_{\mathbb{R}} h_{\mathbf{x}}(t)\, h_{\mathbf{x}'}(t)\, \mathrm{d}t \\ &= \int_{\mathbb{R}} \exp\left(-\frac{(t - \mathbf{x})^2}{2\ell(\mathbf{x})^2} - \frac{(t - \mathbf{x}')^2}{2\ell(\mathbf{x}')^2}\right) \mathrm{d}t. \end{aligned} \tag{21}$$

Setting $a = 1/(2\ell(\mathbf{x})^2)$ and $b = 1/(2\ell(\mathbf{x}')^2)$ and completing the square in t:

$$(t - \mathbf{x})^2 a + (t - \mathbf{x}')^2 b = (a + b)\left(t - \frac{a\mathbf{x} + b\mathbf{x}'}{a + b}\right)^2 + \frac{ab(\mathbf{x} - \mathbf{x}')^2}{a + b}. \tag{22}$$

The Gaussian integral evaluates to $\sqrt{\pi/(a + b)}$, giving

$$k(\mathbf{x}, \mathbf{x}') = \sqrt{\frac{\pi}{a + b}}\, \exp\left(-\frac{ab}{a + b}(\mathbf{x} - \mathbf{x}')^2\right). \tag{23}$$

Substituting back:

$$\frac{ab}{a + b} = \frac{1}{2(\ell(\mathbf{x})^2 + \ell(\mathbf{x}')^2)}, \qquad \frac{\pi}{a + b} = \frac{2\pi\,\ell(\mathbf{x})^2 \ell(\mathbf{x}')^2}{\ell(\mathbf{x})^2 + \ell(\mathbf{x}')^2}. \tag{24}$$

Normalising $h_{\mathbf{x}}$ to unit L_2 norm (so that $k(\mathbf{x}, \mathbf{x}) = 1$ when $\sigma_f^2 = 1$) absorbs the prefactor and yields exactly (18).

Interpretation. Each term of the Gibbs kernel has a clear role:

- $\exp(-\|\mathbf{x}-\mathbf{x}'\|^2/(\ell(\mathbf{x})^2+\ell(\mathbf{x}')^2))$: the decay of correlation with distance, where the effective length scale is the root-mean-square of the local lengthscales $\ell(\mathbf{x})$ and $\ell(\mathbf{x}')$.
- $(2\ell(\mathbf{x})\ell(\mathbf{x}')/(\ell(\mathbf{x})^2 + \ell(\mathbf{x}')^2))^{1/2}$: an amplitude discount when $\ell(\mathbf{x}) \neq \ell(\mathbf{x}')$. Two points with very different local lengthscales represent different "resolution" regimes; even if they are spatially close, their process realizations are poorly correlated because their generating filters have different bandwidths.

Remark 5.1 (Reduction to RBF)**.** When $\ell(\mathbf{x}) = \ell(\mathbf{x}') = \ell_0$ the amplitude prefactor equals 1 and the exponent becomes $-\|\mathbf{x} - \mathbf{x}'\|^2/\ell_0^2$, recovering (17) (with the convention that the RBF uses ℓ_0 not $\ell_0/\sqrt{2}$). Thus the Gibbs kernel strictly generalises the RBF.

Remark 5.2 (Positive definiteness)**.** Because $k_G(\mathbf{x}, \mathbf{x}') = \langle h_{\mathbf{x}}, h_{\mathbf{x}'} \rangle_{L^2}$, it is an inner product kernel and hence unconditionally positive semi-definite for any choice of $\ell(\cdot) > 0$. This holds regardless of whether ℓ is itself modelled by a GP.

6 Learning the Lengthscale Field

6.1 Two-Layer Architecture

The full forecasting system consists of two coupled nested kriging layers:

Layer 1 (LengthscaleField): Estimates $\ell(\mathbf{x})$ for all $\mathbf{x} \in \mathbb{R}^L$ by fitting RBF models block-by-block and interpolating via a meta-GP.

Layer 2 (NestedKrigingForecaster): Uses the estimated $\hat{\ell}(\mathbf{x})$ to evaluate the Gibbs kernel and runs the nested kriging aggregation in Section 4.

The two layers share the same block partition $\{B_m\}$ so that the lengthscale estimated in block m of Layer 1 directly informs the Gibbs kernel computed for the same block in Layer 2.

6.2 Stage 1: Local Lengthscale Estimation via MLE

Within block m, treat the RBF kernel (17) as the local model with a constant scalar lengthscale $\ell_m \in \mathbb{R}_{>0}$ (shared across all H output dimensions). The Gaussian log-likelihood for block m is

$$\mathcal{L}_m(\ell_m) = -\frac{H}{2}\log\left|\mathbf{K}_m(\ell_m) + \sigma_n^2\mathbf{I}\right| - \frac{1}{2}\operatorname{tr}\Big(\mathbf{Y}_m^\top\left[\mathbf{K}_m(\ell_m) + \sigma_n^2\mathbf{I}\right]^{-1}\mathbf{Y}_m\Big), \tag{25}$$

where $[\mathbf{K}_m(\ell_m)]_{ij} = \sigma_f^2 \exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/\ell_m^2)$ for $\mathbf{x}_i, \mathbf{x}_j \in B_m$.

What $\mathbf{K}_m(\ell_m)$ means. $\mathbf{K}_m(\ell_m)$ is an $n_m \times n_m$ matrix whose (i, j) entry is the RBF covariance between the i-th and j-th context windows inside block m, computed with the scalar lengthscale ℓ_m. As $\ell_m \to \infty$, all entries approach σ_f^2 and the matrix degenerates (all windows look identical). As $\ell_m \to 0$, the matrix approaches $\sigma_f^2\mathbf{I}$ and each window is treated as uncorrelated.

Why the trace equals a sum over horizons. Expanding by columns of $\mathbf{Y}_m = [\mathbf{y}_{m,1} \cdots \mathbf{y}_{m,H}]$:

$$\operatorname{tr}\Big(\mathbf{Y}_m^\top\mathbf{K}_m^{-1}\mathbf{Y}_m\Big) = \sum_{h=1}^{H}\mathbf{y}_{m,h}^\top\mathbf{K}_m^{-1}\mathbf{y}_{m,h}, \tag{26}$$

where $\mathbf{y}_{m,h} \in \mathbb{R}^{n_m}$ is the h-th-step forecast target for all windows in block m. Each term is the quadratic form of a single GP regression. The log-determinant term acquires a factor H

because the same covariance matrix governs all H outputs; crucially, the factor H does not affect $\arg\max_{\ell_m}$ and can be dropped for optimisation.

The optimisation

$$\hat{\ell}_m = \arg\max_{\ell_m \in [\ell_{\min}, \ell_{\max}]} \mathcal{L}_m(\ell_m) \tag{27}$$

is one-dimensional and solved by golden-section search (no gradient required, no external dependencies). Bounds $[\ell_{\min}, \ell_{\max}]$ are set in log-space as $[\exp(\texttt{LOG_ELL_LO}), \exp(\texttt{LOG_ELL_HI})]$ with recommended defaults $\texttt{LOG_ELL_LO} = \log 1.0$ and $\texttt{LOG_ELL_HI} = \log(L/3)$, keeping the effective lengthscale below one-third of the context window dimension.

Remark 6.1 (Constant predictions and over-smoothing)**.** If $\ell_{\max}$ is set too large (e.g. $e^5 \approx 148$ for $L = 48$), the MLE will often select $\ell_m \approx \ell_{\max}$ because a very long lengthscale trivially reduces the log-determinant while the quadratic form is near-constant. This causes all predictions to equal the training mean – a warning sign that `LOG_ELL_HI` needs tightening.

6.3 Stage 2: Meta-GP over Block Centroids

After Stage 1 we have M pairs $\{(\bar{\mathbf{x}}_m, \hat{\ell}_m)\}_{m=1}^M$. We model $g(\mathbf{x}) = \log \ell(\mathbf{x})$ (working in log-space enforces positivity) as a GP:

$$g(\mathbf{x}) \sim \mathcal{GP}\big(0,\, k_{\text{SE}}(\cdot, \cdot;\, \ell_{\text{meta}}, \sigma_{\text{meta}})\big). \tag{28}$$

The meta-GP has its own hyper-lengthscale ℓ_{meta} (governing how smoothly $\ell(\mathbf{x})$ varies over the window space) and amplitude σ_{meta}. These are fixed by a second MLE over the M log-lengthscale observations $\hat{g}_m = \log \hat{\ell}_m$.

The meta-GP treats the block-level estimates $\hat{g}_m$ as heteroscedastic observations:

$$\hat{g}_m = g(\bar{\mathbf{x}}_m) + \delta_m, \quad \delta_m \sim \mathcal{N}(0, \sigma^2_{\delta,m}), \tag{29}$$

where $\sigma^2_{\delta,m}$ is a per-block noise variance estimated from the local MLE residuals (capturing how reliably block m could estimate its own ℓ_m).

The posterior predictive distribution at any query $\mathbf{x}^*$ is Gaussian with mean $\hat{g}(\mathbf{x}^*)$ and variance $v(\mathbf{x}^*)$, computed by standard GP formulae applied to the M centroid inputs and M log-lengthscale targets.

6.4 Stage 3: Jensen-Corrected Lengthscale Prediction

Because we model $g = \log \ell$ and the posterior over g is Gaussian, the posterior over $\ell = e^g$ is log-normal. The *mean* of this log-normal is

$$\tilde{\ell}(\mathbf{x}^*) = \mathbb{E}[\ell(\mathbf{x}^*)] = \exp\big(\hat{g}(\mathbf{x}^*) + \tfrac{1}{2} v(\mathbf{x}^*)\big)\,, \tag{30}$$

where the $v(\mathbf{x}^*)/2$ term is the Jensen correction that accounts for the curvature of the exponential. This is used as the effective lengthscale at query $\mathbf{x}^*$ when evaluating the Gibbs kernel in Layer 2.

Remark 6.2 (Why not just exponentiate the posterior mean?)**.** Using $\exp(\hat{g}(\mathbf{x}^*))$ would systematically underestimate the expected lengthscale wherever $v(\mathbf{x}^*)$ is large (i.e. at query points far from any centroid). The Jensen correction (30) properly accounts for posterior uncertainty and prevents the model from being overconfident about short lengthscales in data-sparse regions of the window space.

6.5 Using the Lengthscale Field in the Gibbs Kernel

Once $\tilde{\ell}(\mathbf{x})$ is available, the Gibbs kernel (18) is evaluated as follows. For two windows $\mathbf{x}_i, \mathbf{x}_j$ within block m:

$$k_G(\mathbf{x}_i, \mathbf{x}_j) = \sigma_f^2 \left(\frac{2\,\tilde{\ell}(\mathbf{x}_i)\,\tilde{\ell}(\mathbf{x}_j)}{\tilde{\ell}(\mathbf{x}_i)^2 + \tilde{\ell}(\mathbf{x}_j)^2} \right)^{1/2} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\tilde{\ell}(\mathbf{x}_i)^2 + \tilde{\ell}(\mathbf{x}_j)^2} \right). \tag{31}$$

For the second-layer centroid-to-centroid kernel $\mathbf{C}$ and the centroid-to-query vector $\boldsymbol{c}(\mathbf{x}^*)$, the same formula is used with the predicted lengthscales at the respective inputs.

Remark 6.3 (Within-block constant lengthscale)**.** Within a single block, all n_m windows share the same MLE-estimated lengthscale $\hat{\ell}_m$. When $\tilde{\ell}(\mathbf{x}_i) = \tilde{\ell}(\mathbf{x}_j) = \hat{\ell}_m$, the Gibbs kernel reduces exactly to the RBF with lengthscale $\hat{\ell}_m$. Adaptivity thus operates *between* blocks, not within them; finer within-block adaptivity would require smaller blocks at the cost of less data per MLE.

7 Missing and Unobserved Data

7.1 Types of Missingness in Time Series

We distinguish two operationally distinct scenarios:

1. **Sporadic missing values:** NaN values scattered throughout an otherwise continuous series (e.g. sensor dropouts, data collection gaps). The series is conceptually continuous; the missingness is an artefact of data collection.
2. **Intermittent demand:** Most observations are exactly zero, with occasional non-zero "demand" events. The zeros are structural, not missing; the forecasting objective is to model the count/occurrence jointly with the demand size.

7.2 Sporadic Missingness: Linear Imputation and Filtering

Context imputation. A context window $\mathbf{x}_t$ drawn from a series with NaN entries may contain missing values. Before the window is used (either as a training input or a test query), each missing element is imputed by linear interpolation between its nearest observed neighbours within the window. Boundary missing values are filled by the nearest observed value (constant extrapolation). This ensures every context vector is a dense element of $\mathbb{R}^L$ with no arithmetic special cases.

Training window filtering. Imputed targets $\mathbf{f}_t$ are only included in $\mathcal{D}$ if the observed fraction of the target window exceeds a minimum threshold $\tau \in (0, 1]$:

$$\text{include window} \iff \frac{\#\{h : y_{t+h-1} \text{ is observed}\}}{H} \geq \tau. \tag{32}$$

A threshold of $\tau = 0.5$ excludes windows where more than half the forecast horizon is unobserved, avoiding training on heavily imputed targets that could mislead the GP.

Safe Cholesky. After imputation, local block matrices $\mathbf{K}_m + \sigma_n^2 \mathbf{I}$ may still be numerically near-singular due to duplicate or near-duplicate windows in a block. A jitter loop adds $\delta \mathbf{I}$ with $\delta = 10^{-k}$ for $k = 6, 5, \ldots, -8$ (increasing jitter) until the Cholesky succeeds.

7.3 Intermittent Demand: Two-Model Approach

For datasets with high zero-inflation (e.g. `car_parts_with_missing`), a single GP regression is inappropriate because the target distribution is a mixture: a point mass at zero (no demand) and a positive continuous distribution (demand occurs). We use a two-model approach:

Model 1 (BernoulliKrigingClassifier): Predicts the probability p_h^* that demand is non-zero at each horizon h.

Model 2 (NestedKrigingForecaster on non-zero windows): Predicts the demand level q_h^* conditional on demand being positive.

The final point forecast is

$$\hat{y}_{t+h} = p_h^* \cdot q_h^*, \tag{33}$$

which integrates out the occurrence uncertainty.

Bernoulli Kriging Classifier. The classifier uses the same window pooling and nested kriging architecture as the regression model, but with binary targets $z_t^{(h)} = \mathbf{1}[y_{t+h} > 0]$. The GP output is passed through a sigmoid to obtain p_h^*. Because exact Gaussian inference is not appropriate for binary targets, we use a Laplace approximation: find the mode of $p(\boldsymbol{f}|\mathbf{z}, \mathbf{X})$ and approximate the posterior as a Gaussian centred there.

Laplace approximation for the binary GP. Let $\boldsymbol{f}_m \in \mathbb{R}^{n_m}$ be the latent function values in block m and $\mathbf{z}_m \in \{0,1\}^{n_m}$ the binary targets. The Bernoulli log-likelihood is

$$\ell(\boldsymbol{f}_m) = \sum_{i \in B_m} \big[z_i \log \sigma(f_i) + (1 - z_i) \log(1 - \sigma(f_i)) \big], \tag{34}$$

where $\sigma(t) = (1 + e^{-t})^{-1}$. The Laplace mode $\hat{\boldsymbol{f}}_m$ is found by Newton iterations, and the posterior covariance is approximated as $(\mathbf{K}_m^{-1} + \boldsymbol{\Lambda})^{-1}$ where $\boldsymbol{\Lambda} = \mathrm{diag}(\sigma(\hat{f}_i)(1 - \sigma(\hat{f}_i)))$.

8 Multivariate Time Series

8.1 Problem Setup

A multivariate series of dimension p is a sequence $\{(y_t^{(1)}, \ldots, y_t^{(p)})\}_{t=1}^T$ where each $y_t^{(i)}$ is a scalar observation of channel i. The target is a joint $p \times H$ forecast:

$$\hat{\mathbf{F}}^* = \begin{pmatrix} \hat{f}_{1,1}^* & \cdots & \hat{f}_{1,H}^* \\ \vdots & & \vdots \\ \hat{f}_{p,1}^* & \cdots & \hat{f}_{p,H}^* \end{pmatrix} \in \mathbb{R}^{p \times H}. \tag{35}$$

8.2 Channel-by-Channel Window Pooling

Rather than concatenating all p channels into a single pL-dimensional context (which would scale poorly for large p such as $p = 21$ for Jena weather), we treat each channel independently. For channel i, the L-step context is

$$\mathbf{x}_t^{(i)} = (y_{t-L}^{(i)}, \ldots, y_{t-1}^{(i)}) \in \mathbb{R}^L, \qquad \mathbf{f}_t^{(i)} = (y_t^{(i)}, \ldots, y_{t+H-1}^{(i)}) \in \mathbb{R}^H. \tag{36}$$

All channels from all series are pooled into a shared context matrix:

$$\mathbf{X}_{\text{all}} \in \mathbb{R}^{N_{\text{total}} \times L}, \qquad N_{\text{total}} = N_{\text{windows}} \times p, \tag{37}$$

along with a channel index $c_i \in \{1, \ldots, p\}$ recorded in the window metadata. This shared pool is used for the LengthscaleField (Stage 1–2–3 above), so the estimated $\tilde{\ell}(\mathbf{x})$ captures dynamics common to all channels.

Remark 8.1 (Why share the lengthscale field across channels?)**.** All p channels of a multivariate series share the same *time index*. A context window for channel i at time t encodes the same temporal structure (trend, season, recent shock) as channel j at the same t, just scaled differently. Pooling windows from all channels gives p times as many anchors for the meta-GP, producing a more reliable and spatially dense lengthscale field. Per-channel fields would have p times fewer anchors each, increasing meta-GP uncertainty.

8.3 Per-Channel NestedKrigingForecasters

After the shared LengthscaleField is fitted, each channel $i \in \{1, \ldots, p\}$ gets its own `NestedKrigingForecaster` fitted only on the windows with $c_j = i$:

$$\mathcal{D}_i = \{(\mathbf{x}_j^{(i)}, \mathbf{f}_j^{(i)}) : c_j = i\} \subset \mathcal{D}. \tag{38}$$

The per-channel forecasters use the *same* LengthscaleField instance to evaluate $\tilde{\ell}(\mathbf{x})$, ensuring that the Gibbs kernel reflects the shared temporal structure. Only the second-layer regression targets differ between channels.

8.4 MultivariateNestedKrigingForecaster

The `MultivariateNestedKrigingForecaster` is a wrapper that:

1. Fits one shared `LengthscaleField` on $\mathbf{X}_{\text{all}}$ and $\mathbf{Y}_{\text{all}}$.
2. Fits p channel-specific `NestedKrigingForecaster` instances.
3. At prediction time for a query (p, L) context matrix $\mathbf{Q}$, routes row $\mathbf{q}^{(i)} \in \mathbb{R}^L$ to forecaster i and stacks the results:

$$\hat{\mathbf{F}}^*(\mathbf{Q}) = \begin{pmatrix} \texttt{forecaster}_1.\mathit{predict}(\mathbf{q}^{(1)}) \\ \vdots \\ \texttt{forecaster}_p.\mathit{predict}(\mathbf{q}^{(p)}) \end{pmatrix} \in \mathbb{R}^{p \times H}. \tag{39}$$

Context size heuristic for large p. For large p, the pooled dataset $\mathcal{D}$ grows rapidly. To keep the NK computation tractable, the context length L is shortened as p grows:

$$\begin{aligned} p \leq 2 &: L = \min(200,\, \max(2H, 48)), \\ p \leq 7 &: L = \min(200,\, \max(2H, 32)), \\ p > 7 &: L = \min(200,\, \max(H, 24)). \end{aligned}$$

This balances context richness against the curse of dimensionality in the context space.

Missing data in multivariate series. When any channel has NaN values, the same linear imputation and window-filtering strategy from Section 7 is applied per channel before pooling. Channels are imputed independently; there is no cross-channel imputation.

9 Cross-Dataset Nested Kriging

9.1 Motivation and Challenge

The within-dataset model in Sections 4–8 builds one nested kriging predictor per dataset. For rare or short series, the number of pooled windows N may be too small to estimate a reliable lengthscale field, and the second-layer $M \times M$ system is under-constrained. Cross-dataset kriging addresses this by transferring information from related datasets at different frequencies or from similar domains.

The fundamental obstacle is that windows from different datasets live in *incompatible* input spaces. A monthly series with $L_M = 48$ months of context has windows in $\mathbb{R}^{48}$, while a daily series with $L_D = 30$ days of context has windows in $\mathbb{R}^{30}$. No well-defined inner product exists between $\mathbb{R}^{48}$ and $\mathbb{R}^{30}$; the Gibbs kernel (18) requires both arguments to live in the same space.

9.2 Feature Map Approach (Warped Kernel)

The standard resolution is to map all context windows to a common d-dimensional representation via a *feature map* $\phi : \mathbb{R}^L \to \mathbb{R}^d$:

$$k_\phi(\mathbf{x}, \mathbf{x}') = k\big(\phi(\mathbf{x}),\, \phi(\mathbf{x}')\big), \tag{40}$$

where $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is any kernel on the shared embedding space. The cross-dataset nested kriging predictor then uses k_ϕ in place of k_G throughout Sections 4–6, with the same two-layer block structure.

Candidate feature maps include:

- **Normalised lag embeddings:** Map each window to d normalised sample autocorrelations $\hat{\rho}_k = \hat{r}_k/\hat{r}_0$ for $k = 1, \ldots, d$. The ACF is defined in time units, so it is frequency-agnostic.
- **Time2Vec or Fourier features:** Embed periodicity structure explicitly through basis functions at multiple frequencies.
- **Summary statistics:** $\phi(\mathbf{x}) = (\text{mean}, \text{std}, \text{trend slope}, \text{seasonal amplitude}, \ldots)$.

The blurring trade-off. Any feature map that compresses $\mathbb{R}^L \to \mathbb{R}^d$ with $d < L$ necessarily discards information. Two windows that are distinct in $\mathbb{R}^L$ may collide in $\mathbb{R}^d$, causing the predictor to assign equal weight to genuinely different temporal patterns. This *blurring* worsens as L grows and d stays fixed. The cross-dataset model therefore achieves lower within-dataset accuracy than the full L-dimensional Gibbs model; its value lies in stabilising predictions for data-sparse series.

9.3 Class-Level Intermediate Aggregation

Some GIFT-Eval dataset families measure the same physical quantity at different temporal frequencies. Examples include `US_births/M`, `US_births/W`, `US_births/D` (daily, weekly, and monthly birth counts) and analogous families for electricity consumption, air traffic, and COVID statistics.

For such *class-homogeneous* families, a dedicated intermediate aggregation layer is mathematically well-motivated when:

1. Resolutions within the class have *asymmetric data coverage*: e.g. the monthly series spans 30 years while the daily series spans only 2, so the monthly data carries long-range trend information not visible to the daily model.
2. A *temporal aggregation operator* links the resolutions: $y_{\text{weekly}} = \sum_{d \in \text{week}} y_{\text{daily},d}$. This operator defines a principled bridge between the two window spaces.

Normalised-lag resolution. Rather than defining cross-resolution covariances directly (which faces the same incompatible-space problem), the recommended approach is to *harmonise* all series in the class to a common frequency (the finest available) before pooling windows. Monthly values are repeated or interpolated to the daily resolution, daily windows are pooled together with the native daily windows, and the combined pool is fed into a single nested kriging model. The result is a class-level model in a single, well-defined window space.

This approach is preferable to adding a third nested kriging layer with cross-resolution covariances because:

- No new kernel construction is required.
- The blurring problem does not arise (all windows are in the same $\mathbb{R}^L$).
- When coverage is symmetric across resolutions, the coarser series add no information (daily already implies weekly), and the harmonisation step simply does not include them.

When coverage *is* asymmetric, the normalised-lag harmonisation effectively gives the daily model access to the long historical trend from the monthly series, achieving the information-sharing goal of the intermediate class layer without its computational complexity.

9.4 Full Cross-Dataset Architecture

The complete four-layer architecture for cross-dataset inference is:

1. **Within-series layer:** Pool windows from each individual series using the series-specific L. Normalise per series.
2. **Within-dataset layer:** Fit a LengthscaleField and NestedKrigingForecaster using all pooled windows from that dataset. This is the primary model; it operates in $\mathbb{R}^L$ and uses the full Gibbs kernel.
3. **Class-level layer (optional):** For homogeneous families at different frequencies, harmonise to a common frequency and pool across the class before running a class-level nested kriging model. Only beneficial when coverage is asymmetric.
4. **Cross-class layer:** Map all dataset-level predictions to a shared feature space $\mathbb{R}^d$ via ϕ. Run a final nested kriging aggregation in $\mathbb{R}^d$. The per-dataset predictions serve as "observations" in the meta-GP; the second-layer noise captures how much each dataset's predictions deviate from the global pattern.

The within-dataset predictions (Layer 2) are always the primary forecast; the cross-class layer (Layer 4) provides a regularised prior that smooths extreme predictions for data-sparse datasets and transfers patterns from data-rich datasets in the same domain.

10 Summary and Notation Index

10.1 Summary of the Full Method

Table 1 summarises the key design choices at each layer.

Layer	Input	Kernel	Blocks	Output
LengthscaleField (Stage 1)	$\mathbf{X}_{\text{all}} \in \mathbb{R}^{N \times L}$	RBF (constant ℓ_m per block)	M	$\{\hat{\ell}_m\}$
LengthscaleField (Stage 2–3)	$\{\bar{\mathbf{x}}_m, \hat{\ell}_m\}$	RBF (meta-GP)	1	$\tilde{\ell}(\cdot)$
NestedKrigingForecaster	$\mathbf{X} \in \mathbb{R}^{N \times L}$	Gibbs with $\tilde{\ell}$	M	$\hat{\mathbf{f}}^* \in \mathbb{R}^H$
(Multivariate) p per-channel	$\mathbf{X}_i \in \mathbb{R}^{N_i \times L}$	Gibbs (shared $\tilde{\ell}$)	M_i	$\hat{\mathbf{F}}^* \in \mathbb{R}^{p \times H}$
Cross-dataset (future)	$\phi(\mathbf{X}) \in \mathbb{R}^{N \times d}$	any kernel on $\mathbb{R}^d$	M_{cross}	global $\hat{\mathbf{f}}^*$

Table 1: Architecture overview. All layers use the Rullière et al. (2018) nested kriging aggregation formula (15).

10.2 Notation Index

T	Length of a time series
L	Context (look-back) length
H	Forecast horizon
p	Number of channels (1 for univariate)
N	Total number of pooled training windows
M	Number of blocks in the nested kriging partition
n_m	Number of windows in block m
$\bar{\mathbf{x}}_m$	Centroid of block m
$k(\cdot, \cdot)$	Kernel function
$\mathbf{K}_m$	Local kernel matrix for block m, shape $n_m \times n_m$
$\mathbf{C}$	Between-centroid kernel matrix, shape $M \times M$
$\mathbf{D}$	Diagonal matrix of local prediction variances at centroids
σ_n^2	Observation noise variance
σ_f^2	Prior signal variance
$\ell(\mathbf{x})$	Lengthscale field evaluated at $\mathbf{x}$
$\hat{\ell}_m$	MLE lengthscale for block m
$g(\mathbf{x})$	Log-lengthscale: $g = \log \ell$
$\hat{g}(\mathbf{x})$, $v(\mathbf{x})$	Posterior mean and variance of log-lengthscale at $\mathbf{x}$
$\tilde{\ell}(\mathbf{x})$	Jensen-corrected predicted lengthscale: $e^{\hat{g}(\mathbf{x})+v(\mathbf{x})/2}$
$\phi : \mathbb{R}^L \to \mathbb{R}^d$	Feature map for cross-dataset embedding
τ	Minimum observed fraction for training window inclusion

10.3 References

- D. Rullière, N. Durrande, F. Bachoc, C. Chevalier (2018). *Nested Kriging predictions for datasets with a large number of observations.* Statistics and Computing, 28(4), 849–867.
- M. N. Gibbs (1997). *Bayesian Gaussian Processes for Regression and Classification.* PhD thesis, University of Cambridge.
- C. E. Rasmussen and C. K. I. Williams (2006). *Gaussian Processes for Machine Learning.* MIT Press.